SIE 433 Fundamentals of Data Science for Engineers

Exam I Graduate

8:00, March 29, 2025 – 23:59, March 30, 2025

NOTES:

1. Materials (lecture notes, sample codes, homework, and solutions) provided in class are allowed. Seek help from other students/internet are not permitted.
2. Except for Question 3, all problems should be solved manually. You may use your own paper to write your answers, but please clearly label each problem number. For Question 3, if you choose to use code, please attach it as an appendix.
3. Please ensure to compile all your solutions into a **single PDF file** and submit it via D2L before the deadline.
4. The test is worth 100 points. The assigned points for each question are given beside it.
5. There are 11 pages in this exam paper.
6. Sign the Honor Code below:

I have neither given nor received aid on this examination, nor have I concealed any violation of the Honor Code

Name (print): Cheran Wong

Student ID: 23701892.

Signature, Date: Cheran Wong 3/29/25.

==========

Question 1: (27 points)

Suppose a group has 4 students (A, B, C, D) with the test scores (with a scale of 0 to 100) and the evaluations listed as follows:

	Test 1	Test 2	Test 3	Test 4	Test 5	Evaluation 1	Evaluation 2
A	76	78	80	79	73	1	1
B	47	58	55	58	58	0	0
C	82	77	97	63	74	0	1
D	64	53	72	84	75	1	0

a) Student B lost his/her score for Test 3 due to some errors. Though there may exist several methods for filling it, identify the MOST appropriate method and fill the blank based on student B's scores in Test 1&2&4&5. (**Note**: the following problems are based on the new B you filled) (3 points)

b) Using Min-max normalization to find the new value for 75 of student A, the new interval is [2, 4]. (8 points)

c) Identify the outliers for student C based on Test 1&2&3&4&5 by calculating the 5 numbers for the boxplot. (8 points)

d) Given the two binary asymmetric attributes, i.e., Evaluation 1 and Evaluation 2, calculate the dissimilarity matrix for students A and B. (8 points)

a) Mean? $\frac{47+58+58+58}{4} = 55.25 \simeq 55.$

b). $\frac{73-73}{80-73}(4-2)+2=2$ $\frac{80-73}{80-73}(4-2)+2=4$

$\frac{76-73}{80-73}(4-2)+2=\frac{20}{7}$ $\frac{75-73}{80-73}(4-2)+2=\frac{18}{7}$

$\frac{78-73}{80-73}(4-2)+2=\frac{24}{7}$

$\frac{79-73}{80-73}(4-2)+2=\frac{26}{7}$

c) Max: 97.

Q3: 82

Median: 77.

Q1: 74.

Min: 63

(Additional page for Question 1)

	E1	E2
d). (i) A	1	1
(j) B	0	0.

$$d(A, B) = \frac{0+2}{0+0+2+0} = 1.$$

Question 2: (19 points)

A study investigates the impacts of multiple factors on '*murder rate*' in 50 randomly chosen locations. The four factors considered include:

- Population (unit: K people)
- Income (unit: $K)
- Illiteracy (unit: percentage)
- Frost (unit: days)

Some outputs from python are shown blow:

Population	Income	illiteracy	Frost	Murder
3615	3624	2.1	20	1.47
3650	6315	1.5	12	1.34
2212	4530	1.8	15	1.13
2110	3378	1.9	65	1.28
2118	5114	1.1	20	1.32
8901	10024	3.5	231	6.2

Figure 1 First 6 samples from the data set

	coef	std err	t	P>|t|	[0.025	0.975]
Population	2.326e-05	1.79e-05	1.303	0.199	-1.27e-05	5.92e-05
Income	4.985e-05	6.99e-05	0.713	0.480	-9.09e-05	0.000
Illiteracy	0.3953	0.118	3.348	0.002	0.158	0.633
Frost	0.0002	0.002	0.086	0.932	-0.003	0.004

Figure 2 Estimated linear regression model coefficients

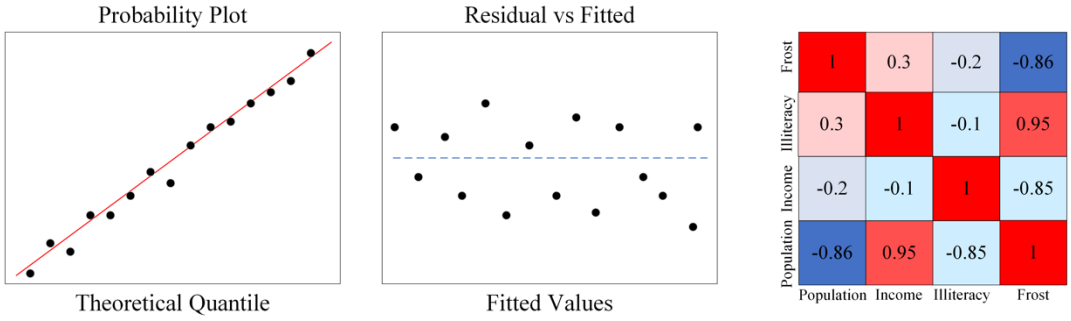


Figure 3 Selected figures from Python output

Based on the given outputs, please answer the following questions and explicitly show your steps of solutions and/or justify your answers.

a) A student writes the Python code to preprocess the csv file called 'murder_rate.csv' as follows (Figure 4). **Follow the comments** in the codes, and find 3 errors in the Python codes and propose your solution to fix those errors (3 points)

```
1. # import necessary libraries
2. import pandas as pd
3. from sklearn import train_test_split
4.
5. # read the csv file
6. murder_rate_df = pd.read_csv('murder_rate.csv')
7.
8.
9. # the dataframe has 5 columns
10. # The first 4 columns are population, income, illiteracy, Frost
11. # The last column is Murder
12. X = murder_rate_df.iloc[:, :]
13. y = murder_rate_df.iloc[:, -1]
14.
15. # The student wants to divide the dataframe such that
16. # the training size is 0.8, and the testing size is 0.2
17. X_train, y_train, X_test, y_test = train_test_split(X, y, random_state = 2023)
```

Figure 4. Please use this figure to answer sub-problem (a)

b) After fixing the error in the code, the student checked the dataframe, and he found that there is an outlier in the dataframe as in Figure 1. Please help him to identify the outlier and give the pandas.Dataframe index for the outlier. (3 points)

c) Given the sample size of the training data set is 50 and given $R_a = 0.8$, please calculate R. (6 points)

d) Please check the model assumptions based on Figure 3 and propose the solution for those unmet assumption. In order to have full assumptions checked, which plot is needed? (4 points)

e) Given the expected complexity vs out-of-sample error curve (blue), the student finds the current model is located as below (Figure 5), what would you suggest him do in terms of the number of variables? (3 points)

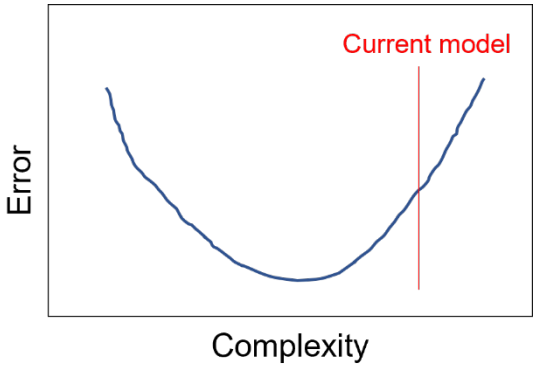


Figure 5. Please use this figure to answer sub-problem (e)

a). Do train, test split first.

```
train, test = train_test_split(murder_state_df,
                               test_size = 0.2,
                               random_state = 2023).

X_train = train.iloc[:, :-1]
y_train = train["Murder"]
X_test = test.iloc[:, :-1]
y_test = test["Murder"]
```

b). Index 5.

Pop.	8901
Inc.	10024
Ill.	3.5.
Frost	231
Mur.	6.2.

c).

$$R_a^2 = 1-(1-R^2)\frac{n-1}{n-(d+1)}$$

$$0.8^2 = 1-(1-R^2)\frac{50-1}{50-(5+1)}$$

$$0.8^2 = 1-(1-R^2)\frac{49}{44}$$

$$0.8^2 \times \frac{44}{49} = 1-(1-R^2)$$

$$1-\left(0.8^2 \times \frac{44}{49}\right) = 1-R^2$$

$$R^2 = 1-\left(1-0.8^2 \times \frac{44}{49}\right)$$

$$R^2 = 0.5747.$$

$$R = \sqrt{0.5747} = 0.7581.$$

(Additional page for Question 2)

d). Pairplots to check for full assumptions.
Histogram for residuals.

e). Decrease the number of variables.

Question 3: (20 points)

You are given a dataset containing *prostate* cancer data. The dataset consists of eight predictor variables (columns 1-8), $\boldsymbol{X} \in R^8$, and an outcome variable Y (column 9). The last column is a binary indicator, where 'T' represents training set $(n = 67)$ and 'F' represents test set $(\tilde{n} = 30)$. Denote the training set by $\{(\boldsymbol{x}_i, y_i), i = 1, \dots, n\}$ and test set by $\{(\tilde{\boldsymbol{x}}_i, \tilde{y}_i), i = 1, \dots, \tilde{n}\}$.

a) Fit the standard linear regression model, $\hat{f}(x) = \hat{\beta}_0 + \hat{\boldsymbol{\beta}}^T \boldsymbol{x}$, using Ordinary Least Squares (OLS). Report its R^2 value, p-values for each predictor, significant predictors at the level $\alpha = 0.05$, $\text{TrainErr} = \frac{1}{n}\sum_{i=1}^{n}[y_i - \hat{f}(x_i)]^2$, and $\text{TestErr} = \frac{1}{\tilde{n}}\sum_{i=1}^{\tilde{n}}[\tilde{y}_i - \hat{f}(\tilde{\boldsymbol{x}}_i)]^2$. (8 points)

b) Fit linear regression models for **ALL COMBINATIONS** of predictors in $\boldsymbol{X}$. Generate the *AIC value - Degree of Freedom* figure for model selection. Please copy your figure in the answer and report the lowest AIC value and the corresponding variables that are selected. (6 points)

c) Fit linear regression models for **ALL COMBINATIONS** of predictors in $\boldsymbol{X}$. Generate the *BIC value - Degree of Freedom* figure for model selection. Please copy your figure in the answer and report the lowest BIC value and the corresponding variables that are selected. (6 points)

(Additional page for Question 3)

exam1

March 30, 2025

[10]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from sklearn import linear_model
from itertools import combinations
from sklearn.metrics import mean_squared_error

df = pd.read_csv("prostate.csv")

train = df[df["train"] == "T"]
test = df[df["train"] == "F"]

x_train = train[["lcavol", "lweight", "age", "lbph", "svi", "lcp", "gleason",␣
 ↪"pgg45"]]
y_train = train["lpsa"]
x_test = test[["lcavol", "lweight", "age", "lbph", "svi", "lcp", "gleason",␣
 ↪"pgg45"]]
y_test = test["lpsa"]
```

3a) Standard Linear Regression Model using OLS

[11]:
```
linear_regression = linear_model.LinearRegression()
linear_regression.fit(x_train, y_train)

lr = sm.OLS(y_train, x_train)
results = lr.fit()
print(results.summary())

# R^2
print("R^2: ", results.rsquared)

# P-values
print("P-values: ", results.pvalues)

# Significant predictors using 0.05 significance level
significant_predictors = results.pvalues[results.pvalues < 0.05]
```

```python
print("Significant predictors: ", significant_predictors)

# Training and Testing Errors
train_predictions = linear_regression.predict(x_train)
test_predictions = linear_regression.predict(x_test)
train_error = np.mean((y_train - train_predictions) ** 2)
test_error = np.mean((y_test - test_predictions) ** 2)
print("Training Error: ", train_error)
print("Testing Error: ", test_error)
```

```
                            OLS Regression Results
==============================================================================
=======
Dep. Variable:                    lpsa   R-squared (uncentered):
0.941
Model:                            OLS   Adj. R-squared (uncentered):
0.933
Method:                 Least Squares   F-statistic:
117.6
Date:                Sat, 29 Mar 2025   Prob (F-statistic):
2.43e-33
Time:                        22:12:28   Log-Likelihood:
-67.549
No. Observations:                  67   AIC:
151.1
Df Residuals:                      59   BIC:
168.7
Df Model:                           8
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
lcavol         0.5706      0.104      5.463      0.000       0.362       0.780
lweight        0.6461      0.189      3.417      0.001       0.268       1.024
age           -0.0181      0.013     -1.380      0.173      -0.044       0.008
lbph           0.1383      0.066      2.100      0.040       0.007       0.270
svi            0.7414      0.296      2.506      0.015       0.149       1.333
lcp           -0.2068      0.110     -1.887      0.064      -0.426       0.013
gleason        0.0120      0.133      0.090      0.928      -0.254       0.278
pgg45          0.0087      0.005      1.844      0.070      -0.001       0.018
==============================================================================
Omnibus:                        0.837   Durbin-Watson:                   1.710
Prob(Omnibus):                  0.658   Jarque-Bera (JB):                0.382
Skew:                          -0.156   Prob(JB):                        0.826
Kurtosis:                       3.198   Cond. No.                         252.
==============================================================================

Notes:
```

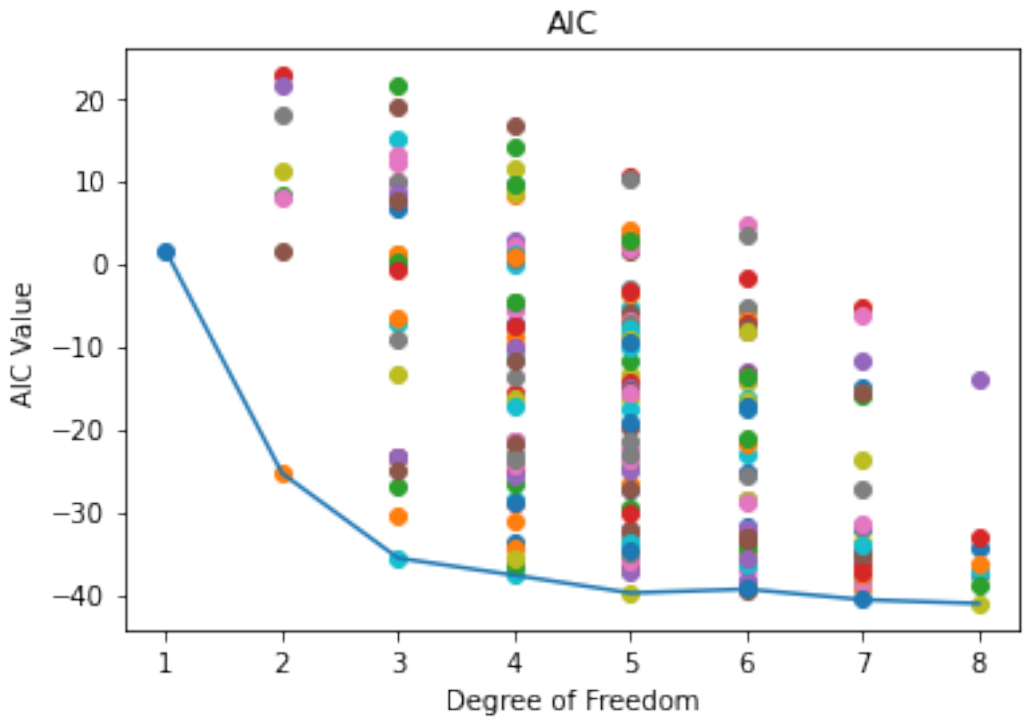
```
[1] R² is computed without centering (uncentered) since the model does not
contain a constant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
R^2:  0.9409776293201828
P-values:  lcavol     9.864471e-07
lweight    1.152215e-03
age        1.726477e-01
lbph       4.001430e-02
svi        1.498475e-02
lcp        6.414204e-02
gleason    9.284580e-01
pgg45      7.020806e-02
dtype: float64
Significant predictors:  lcavol     9.864471e-07
lweight    1.152215e-03
lbph       4.001430e-02
svi        1.498475e-02
dtype: float64
Training Error:  0.4391997680583344
Testing Error:  0.5212740055076022
```

3b) AIC Value vs Degree of Freedom graph

[ ]:
```python
def AIC(train_data, train_label):
    lm = linear_model.LinearRegression().fit(train_data, train_label)
    predict_label = lm.predict(train_data)
    degree_freedom = train_data.shape[1]
    sample_size = train_data.shape[0]
    SSE = np.dot((train_label - predict_label), (train_label - predict_label))
    return sample_size * np.log(SSE / sample_size) + 2 * degree_freedom


feature_names = list(x_train.columns.values)
min_AIC = []
min_AIC.append(y_train.var())
plt.scatter(1, y_train.var())  # when there is only the intercept term, plot␣
 ↪variance
for i in range(1, 8):
    names = list(
        combinations(feature_names, i)
    )  # generate the combination of i feature names and convert it to a list
    print(names)
    aic_list = []  # define a list to store AIC values
    for each_element in names:
        aic_list.append(
            AIC(x_train[list(each_element)], y_train)
        )  # append BIC values to the list
```

```
    min_AIC.append(np.min(aic_list))
    min_index = np.argmin(aic_list)
    print(aic_list)
    for each_value in aic_list:
        plt.scatter(x=i + 1, y=each_value)  # plot AIC values

plt.plot([1, 2, 3, 4, 5, 6, 7, 8], min_AIC)
plt.title("AIC")
plt.xlabel("Degree of Freedom")
plt.ylabel("AIC Value")
plt.show()

print("The minimum AIC values are: ", min_AIC)
print("Minimum AIC value is: ", np.min(min_AIC))
print(
    "The minimum AIC value is obtained using the corresponding features: ",
    names[min_index],
)
```

```
[('lcavol',), ('lweight',), ('age',), ('lbph',), ('svi',), ('lcp',),
('gleason',), ('pgg45',)]
[-25.37360907803762, 8.307136075706351, 22.72785489476331, 21.49302680482578,
1.4198986280740475, 7.9657329011922124, 17.9367573234735, 11.279855240166084]
[('lcavol', 'lweight'), ('lcavol', 'age'), ('lcavol', 'lbph'), ('lcavol',
'svi'), ('lcavol', 'lcp'), ('lcavol', 'gleason'), ('lcavol', 'pgg45'),
('lweight', 'age'), ('lweight', 'lbph'), ('lweight', 'svi'), ('lweight', 'lcp'),
('lweight', 'gleason'), ('lweight', 'pgg45'), ('age', 'lbph'), ('age', 'svi'),
('age', 'lcp'), ('age', 'gleason'), ('age', 'pgg45'), ('lbph', 'svi'), ('lbph',
'lcp'), ('lbph', 'gleason'), ('lbph', 'pgg45'), ('svi', 'lcp'), ('svi',
'gleason'), ('svi', 'pgg45'), ('lcp', 'gleason'), ('lcp', 'pgg45'), ('gleason',
'pgg45')]
[-35.61679636983291, -23.423191382069184, -30.572373525421185,
-26.793741116709395, -23.465477831712484, -23.53103675114468,
-25.057965297652668, 9.772159952599495, 10.026145404472434, -13.35027371992286,
-7.10318826219441, -0.06146891058326265, -6.624520215931504, 21.653294300438926,
0.976837860785988, 8.081948168583008, 19.012534999301877, 12.289955529615469,
-9.169486451203957, 1.075034959457001, 14.94383231948073, 6.53377479177222,
1.023662130072148, 0.18855013616978678, -0.8055907144569368, 8.589135064546184,
7.509944664472728, 13.277806139248092]
[('lcavol', 'lweight', 'age'), ('lcavol', 'lweight', 'lbph'), ('lcavol',
'lweight', 'svi'), ('lcavol', 'lweight', 'lcp'), ('lcavol', 'lweight',
'gleason'), ('lcavol', 'lweight', 'pgg45'), ('lcavol', 'age', 'lbph'),
('lcavol', 'age', 'svi'), ('lcavol', 'age', 'lcp'), ('lcavol', 'age',
'gleason'), ('lcavol', 'age', 'pgg45'), ('lcavol', 'lbph', 'svi'), ('lcavol',
'lbph', 'lcp'), ('lcavol', 'lbph', 'gleason'), ('lcavol', 'lbph', 'pgg45'),
('lcavol', 'svi', 'lcp'), ('lcavol', 'svi', 'gleason'), ('lcavol', 'svi',
'pgg45'), ('lcavol', 'lcp', 'gleason'), ('lcavol', 'lcp', 'pgg45'), ('lcavol',
'gleason', 'pgg45'), ('lweight', 'age', 'lbph'), ('lweight', 'age', 'svi'),
```



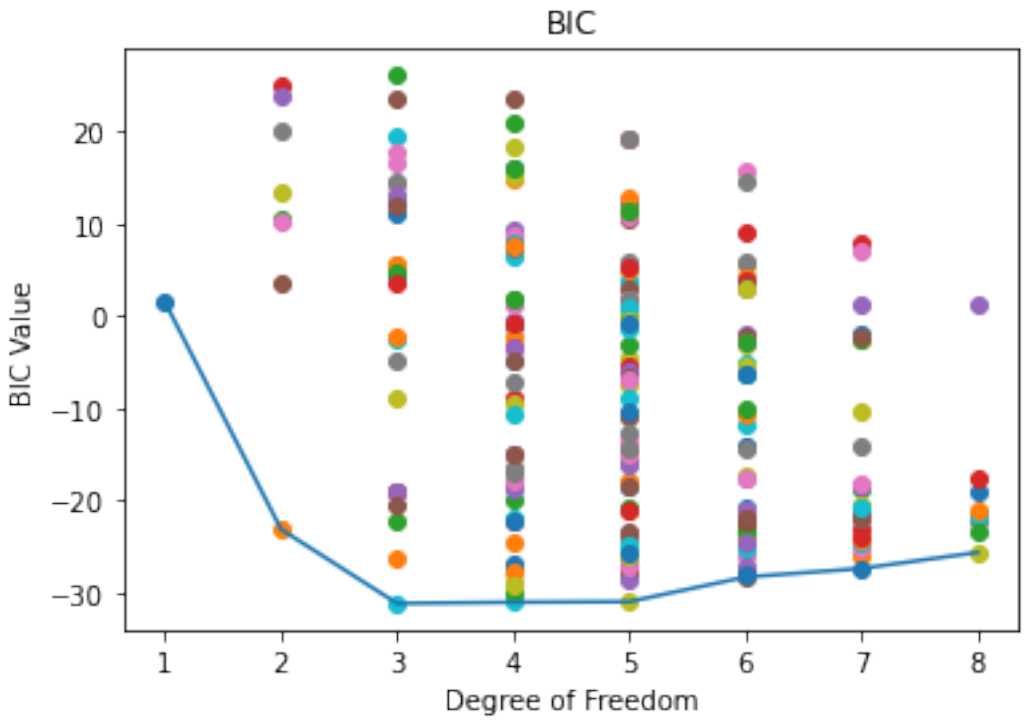

```
The minimum AIC values are:  [1.458809773002296, -25.37360907803762,
-35.61679636983291, -37.68291356508498, -39.82507244977625, -39.364852213098764,
-40.6393931650699, -41.10281207095607]
Minimum AIC value is:  -41.10281207095607
The minimum AIC value is obtained using the corresponding features:  ('lcavol',
'lweight', 'age', 'lbph', 'svi', 'lcp', 'pgg45')
```

3c)

[21]:
```
def BIC(train_data, train_label):
    lm = linear_model.LinearRegression().fit(train_data, train_label)
    predict_label = lm.predict(train_data)
    degree_freedom = train_data.shape[1]
    sample_size = train_data.shape[0]
    SSE = np.dot((train_label - predict_label), (train_label - predict_label))
    return (
        sample_size * np.log(SSE / sample_size) + np.log(sample_size) *␣
 ↪degree_freedom
    )


feature_names = list(x_train.columns.values)
min_BIC = []
min_BIC.append(y_train.var())
```

```
plt.scatter(1, y_train.var())  # when there is only the intercept term, plot␣
 ↪variance
for i in range(1, 8):
    names = list(
        combinations(feature_names, i)
    )  # generate the combination of i feature names and convert it to a list
    print(names)
    bic_list = []  # define a list to store BIC values
    for each_element in names:
        bic_list.append(
            BIC(x_train[list(each_element)], y_train)
        )  # append BIC values to the list
    print(bic_list)
    min_BIC.append(np.min(bic_list))
    for each_value in bic_list:
        plt.scatter(x=i + 1, y=each_value)  # plot BIC values
plt.plot([1, 2, 3, 4, 5, 6, 7, 8], min_BIC)
plt.title("BIC")
plt.xlabel("Degree of Freedom")
plt.ylabel("BIC Value")
plt.show()

print("The minimum BIC values are: ", min_BIC)
print("Minimum BIC value is: ", np.min(min_BIC))
print(
    "The minimum BIC value is obtained using the corresponding features: ",
    names[min_index],
)
```

```
[('lcavol',), ('lweight',), ('age',), ('lbph',), ('svi',), ('lcp',),
('gleason',), ('pgg45',)]
[-23.168916458646656, 10.511828695097318, 24.932547514154276,
23.697719424216746, 3.6245912474650135, 10.170425520583178, 20.141449942864465,
13.48454785955705]
[('lcavol', 'lweight'), ('lcavol', 'age'), ('lcavol', 'lbph'), ('lcavol',
'svi'), ('lcavol', 'lcp'), ('lcavol', 'gleason'), ('lcavol', 'pgg45'),
('lweight', 'age'), ('lweight', 'lbph'), ('lweight', 'svi'), ('lweight', 'lcp'),
('lweight', 'gleason'), ('lweight', 'pgg45'), ('age', 'lbph'), ('age', 'svi'),
('age', 'lcp'), ('age', 'gleason'), ('age', 'pgg45'), ('lbph', 'svi'), ('lbph',
'lcp'), ('lbph', 'gleason'), ('lbph', 'pgg45'), ('svi', 'lcp'), ('svi',
'gleason'), ('svi', 'pgg45'), ('lcp', 'gleason'), ('lcp', 'pgg45'), ('gleason',
'pgg45')]
[-31.207411131050982, -19.013806143287255, -26.162988286639255,
-22.38435587792746, -19.056092592930554, -19.121651512362746,
-20.648580058870735, 14.181545191381428, 14.435530643254364, -8.940888481140929,
-2.693803023412478, 4.347916328198669, -2.2151349771495727, 26.06267953922086,
5.3862230995679194, 12.49133340736494, 23.421920238083807, 16.6993407683974,
-4.760101212422025, 5.484420198238933, 19.353217558262664, 10.943160030554152,
```




```
The minimum BIC values are:  [1.458809773002296, -23.168916458646656,
-31.207411131050982, -31.068835706912083, -31.006301972212388,
-28.341389116143937, -27.4112374487241, -25.66996373521931]
Minimum BIC value is:  -31.207411131050982
The minimum BIC value is obtained using the corresponding features:  ('lcavol',
'lweight', 'age', 'lbph', 'svi', 'lcp', 'pgg45')
```

Question 4 (18 points)

Suppose you are given a dataset with 6 observations, each of which contains only one feature, X_i where $i = 1,2,..,6$. These observations correspond to some true labels y_i. Now, you are using logistic function (with the intercept term) to classify the dataset into two classes, e.g., 0 and 1. Following table provides the necessary information, please answer the following questions.

Observation i	1	2	3	4	5	6
Input variable X_i	1	2	0	6	8	-2
True class y_i	0	1	1	0	1	0
Predicted probability to be 1, $P(\hat{y}_i = 1)$	0.405	0.589	0.208	0.698	0.720	0.139
Predicted class $\hat{y}_i$	0	1	0	1	1	0

1. Fill the blanks for the predicted class $\hat{y}_i$, where $i = 1,2,..,6$, given 0.5 probability threshold (3 points: 0.5 points each)

2. Based on the predicted class and true class, answer the following questions
   a) What is the Manhattan distance between the prediction $\hat{y}_i$ and the ground-truth y_i, where $i = 1,2,..,6$? (3 points)
   b) The logistic regression assumes the log odd is linear. Calculate w_0 and w_1 based on the feature values for observations 1 and 2, i.e., $X_1 = 1$ and $X_2 = 2$. (3 points)

3. Given the predicted class, $\hat{y}_i$ where $i = 1,2,..,6$, that you have filled in subproblem 1,
   a) What is the recall? (3 points)
   b) Calculate the F_1 and F_2 score. Is there any difference? Why? (6 points)

2a). $d = |0-0| + |1-1| + |1-0| + |1-1| + |0-0|$

$= 1.$ #

2b). $P(y=1|x) = \frac{1}{1+e^{-w^T x}}$

$= \frac{1}{1+e^{-(w_0+w_1x_1+w_2x_2)}}$.

$0.405 = \frac{1}{1+e^{-(w_0+w_1)}}$

$1+e^{-(w_0+w_1)} = \frac{1}{0.405}$.

$e^{-(w_0+w_1)} = \frac{1}{0.405} - 1$

$-(w_0+w_1) = \ln\left(\frac{1}{0.405} - 1\right)$.

$-w_1 = 0.38467$

$w_1 = -0.38467$.

$w_0 = 0$

$w_1 = -0.38467$ #

(Additional page for Question 4)

3a). $\text{Recall} = \frac{TP}{TP + FN}$. $TP = 2$, $FN = 1$

$= \frac{2}{3}$.

3b). $F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$. $\text{Precision} = \frac{TP}{TP + FP}$.

$= \frac{2}{3}$.

$= \frac{2 \times \frac{2}{3} \times \frac{2}{3}}{\frac{2}{3} + \frac{2}{3}} = \frac{\frac{8}{9}}{\frac{4}{3}}$.

$= \frac{2}{3}$.

$F2 = \frac{5}{\frac{4}{\text{Prec}} + \frac{1}{\text{Recall}}} = \frac{5}{\frac{4}{\frac{2}{3}} + \frac{1}{\frac{2}{3}}}$.

$= \frac{5}{6 + \frac{3}{2}} = \frac{5}{\frac{15}{2}}$.

$= \frac{2}{3}$.

No difference between F1 and F2 scores. Because # of False Negatives and False Positives are equal.

Question 5: (16 Points, 2 points for each question) Each of the following statements can be TRUE or FALSE. Mark in the "( )" with a "T" is you think it is TRUE, and with an "F" if you think it is FALSE. (No need to show calculation procedures or explanations).

1. (T) Given the off-diagonal values of the 2-by-2 variance-covariance matrix equal to -0.5, the contour for the multivariate normal distribution is an ellipse with a major axis whose direction is $-45°$.

2. (F) The AUC of a random classifier (e.g., random guess), is close to 0.

3. (T) Given the off-diagonal values of the 2-by-2 variance-covariance matrix equal to -0.5, the contour for the multivariate normal distribution is an ellipse with a major axis whose direction is $-45°$.

4. (F) Adjusted R^2 will be increased by adding more predictors.

5. (F) In machine learning, we care more about performance of the algorithm on training dataset than testing dataset.

6. (T) A doctor wants to detect a specific type of cancer as many as possible using logistic regression. It is good for him to put more focus on Precision than the Recall.

7. (F) Logistic regression provides the probability, therefore it is more preferable to use it for predicting the movie length than the linear regression.

8. (T) Gradient descent stops when the difference between $\mathbf{w}_t$ and $\mathbf{w}_{t+1}$ is smaller than a threshold ε. Therefore, it is good to make ε larger in order to get better convergence results.